Assignment 1 - Git & GitHub Fundamentals

Full Name: fatima zahrae el bchiker 155574

GitHub Username: 12005-f

Repository: https://github.com/12005-f/git-session1-12005-f

1. Repository Setup

I copied the supplied STARTER files into git-session1-12005-f and initialized Git with the main branch. My .gitignore excludes local.env and ordinary *.log files, while !important.log allows the important log to be tracked. I verified this using git status --short --ignored.

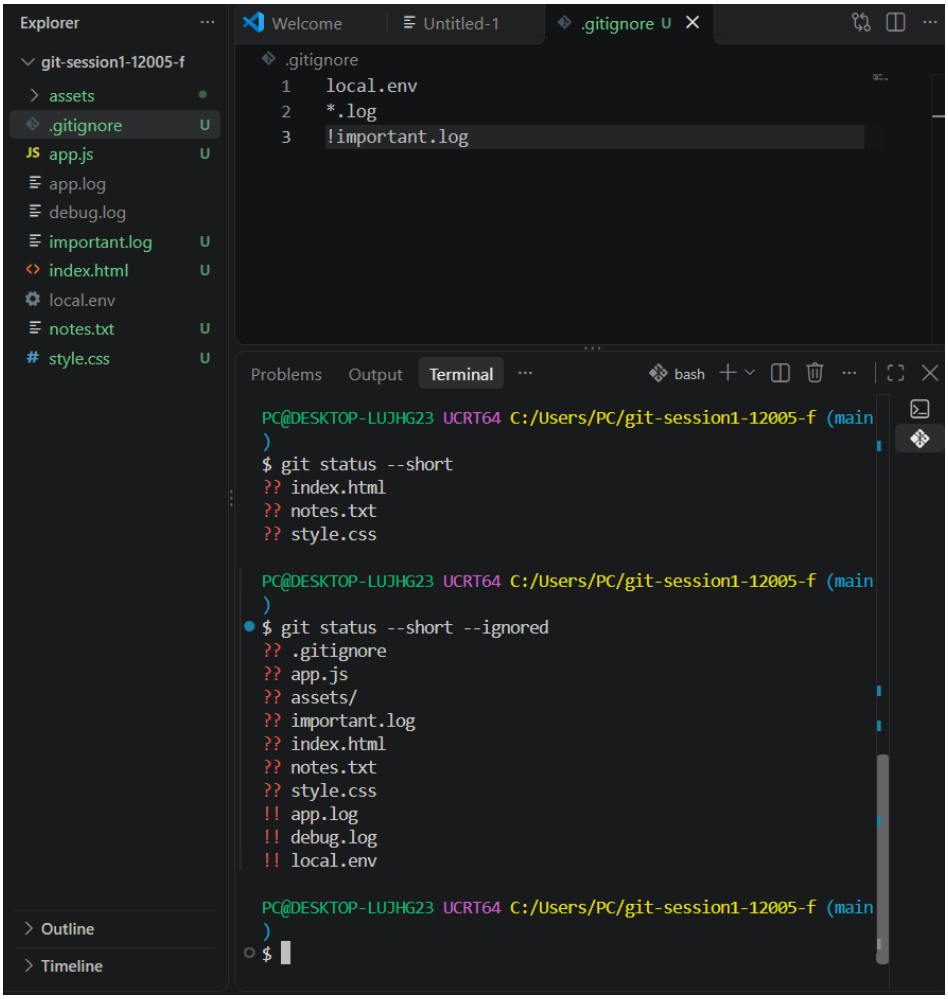


Figure 1. Ignore rules and Git status: ordinary logs and local.env are ignored.

2. README & Commit History

I created README.md with my name, GitHub username, project description, technologies, a Git command example, a documentation link, and the supplied image assets/project-banner.svg. I organized changes into four initial commits: starter project and ignore rules; README and banner; homepage HTML; and homepage CSS. The About feature was committed separately on its branch, satisfying the requirement for at least five meaningful commits across multiple files.

Recorded commits: 8adbcaa (setup), f9bb77a (README), 987615e (HTML), b3573b0 (CSS), e6accda (About); followed by merge c17b48d.

3. Selective Staging

I modified index.html and style.css in the same working session. The screenshot shows both files modified before staging. I then used git add index.html to stage and commit only the HTML change, leaving the CSS change unstaged; afterward I staged and committed CSS separately. This distinguishes the working tree from the staging area.

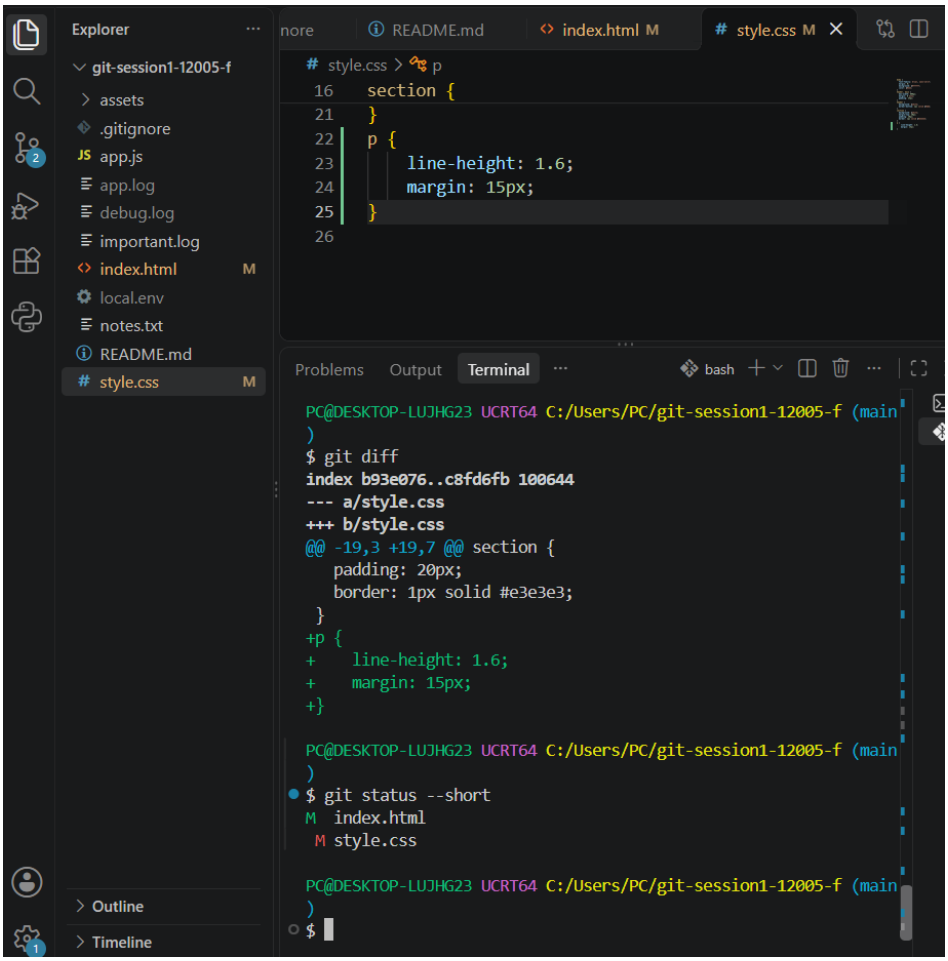


Figure 2. Both index.html and style.css show local modifications before selective staging.

4. Branch & Merge

I created about-feature with git switch -c about-feature, added ABOUT.md, and committed the feature. After switching back to main, I merged it with git merge --no-ff about-feature. The graph confirms a distinct feature commit and merge commit.

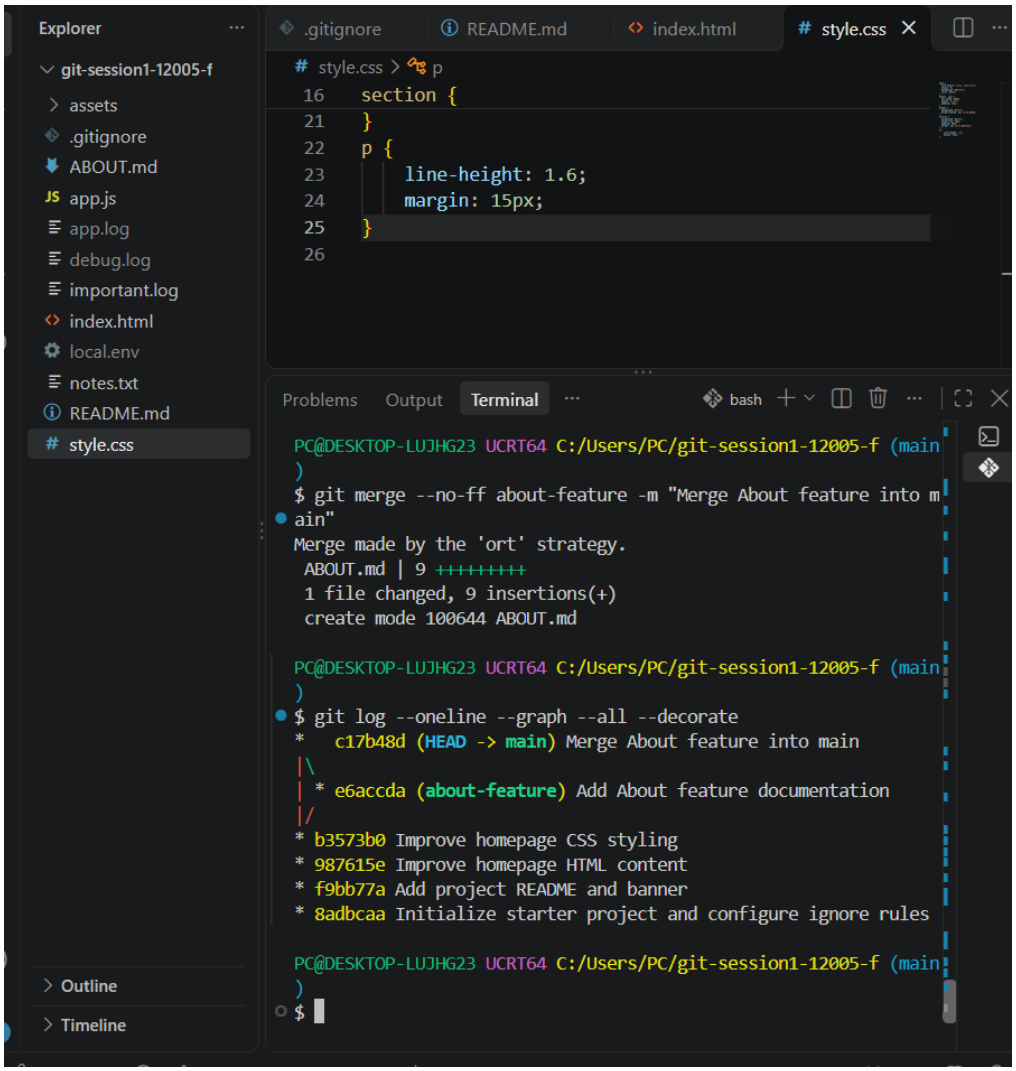


Figure 3. Branch graph showing About feature commit e6accda and merge c17b48d.

5. Remote Changes: Commit / Push and Fetch / Pull

Commit saves staged changes in the local repository; push publishes local commits to GitHub. I connected origin and used git push -u origin main, which confirmed that main tracks origin/main. Fetch downloads remote history without integrating it into local main, whereas pull fetches and integrates remote changes. After a README edit on GitHub, git fetch origin followed by git status showed local main one commit behind origin/main.

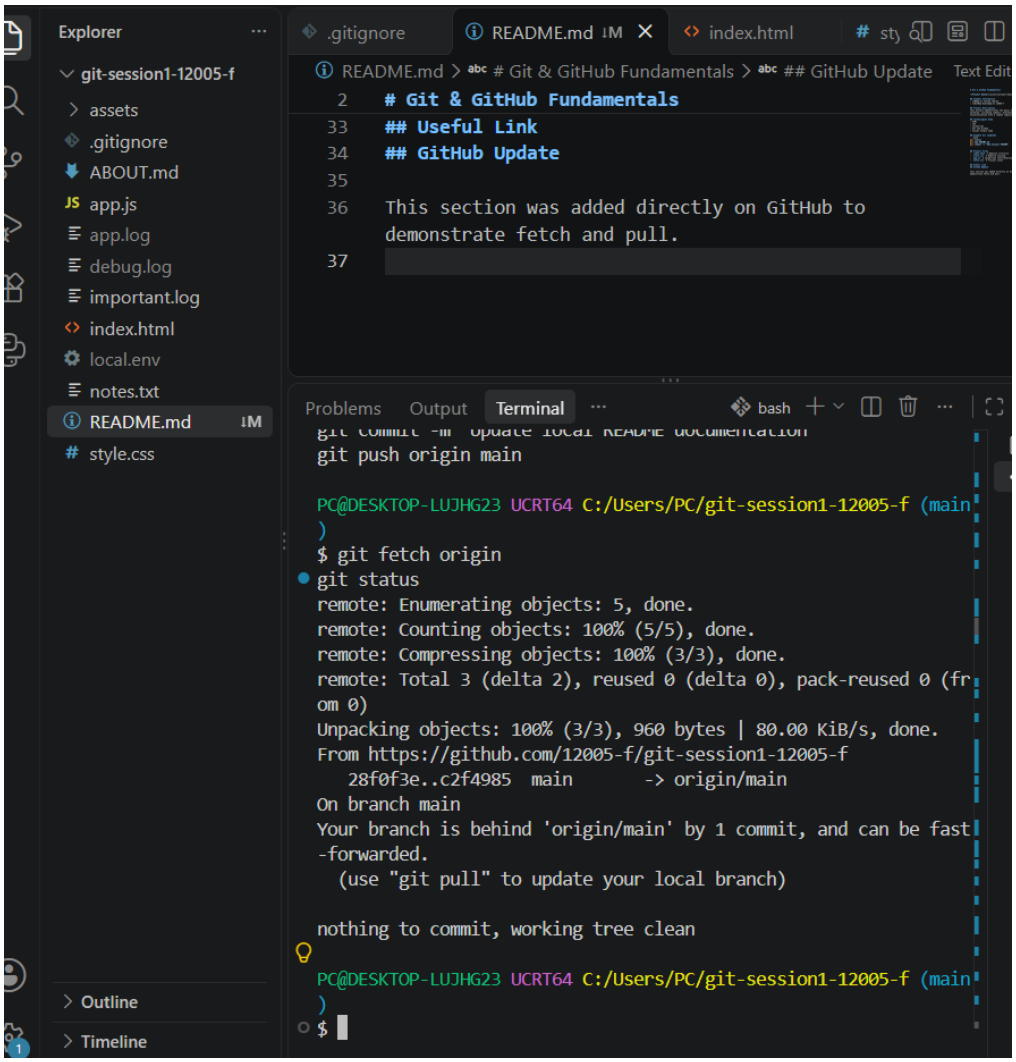


Figure 4. After fetch, local main is one commit behind origin/main.

Verification still needed: The provided screenshots do not show a successful git pull origin main and subsequent up-to-date status. Run the pull and capture that result before final submission.

6. Fair Use of AI

I used AI to help me to create the pdf in good way

7. Sources & Reflection

Sources: Group C Assignment 1 instructions and report template; official Git documentation (https://git-scm.com/docs); my terminal output and GitHub commit history.

The hardest concept was selective staging: changing two files does not mean both must enter the same commit. If Git behaves unexpectedly, I would first check git status and git rev-parse --show-toplevel, then inspect the relevant diff, branch, and remote information.